

Pipeline Lakehouse en tiempo real

Analítica avanzada in-stream con Amazon Redshift

Campo	Detalle
Autor	Andres Capo Plaza
Checkpoint	Pre-entrega 6 – Analítica avanzada in-stream con Redshift
Repositorio	github.com/AndresCapoPlaza/dataops-terraform-aws
Cuenta AWS / Region	010798385513 · us-east-1
Workgroup	lakehouse-wg-dev (Redshift Serverless, 8 RPU)
Fecha de ejecucion	09 de septiembre de 2026

Resumen de resultados

Se procesaron **1.736 eventos** desde Kinesis hacia Redshift con una latencia de punta a punta de **2 segundos en promedio** (maximo 4 s) y **cero errores de casting**: los 1.736 registros parsearon como JSON valido. La consulta federada `v_clicks_hot_vs_cold` combina, para cada producto, los clics en vivo del stream con las 5 ventanas historicas consolidadas por Flink en la tabla Apache Iceberg, leida por Spectrum a traves del Glue Data Catalog.

1. Objetivo y alcance

Este checkpoint incorpora la capa de consulta analítica de baja latencia sobre el pipeline construido en las entregas anteriores. Hasta el modulo 5 el recorrido del dato terminaba en el Lakehouse: Flink consolidaba los clics en ventanas de un minuto y los persistía como tabla Apache Iceberg gobernada por AWS Glue. Ese camino es solido para el analisis historico, pero introduce una latencia de minutos, porque un registro solo es consultable despues de que cierra su ventana y se completa el checkpoint que confirma el commit.

El objetivo aqui es cubrir la franja que ese diseno deja descubierta: **lo que esta pasando ahora mismo**. Redshift Streaming Ingestion conecta el Kinesis Data Stream directamente con el warehouse, sin escalar por S3 ni por un proceso de carga intermedio. El resultado medido en esta implementacion es una latencia de 2 a 4 segundos entre que el evento ocurre y queda disponible para SQL.

La pieza que da valor al conjunto no es ninguna de las dos rutas por separado, sino la posibilidad de consultarlas juntas. Un analista escribe una sola sentencia SQL y obtiene, en la misma fila, el pulso del minuto actual y la serie historica consolidada.

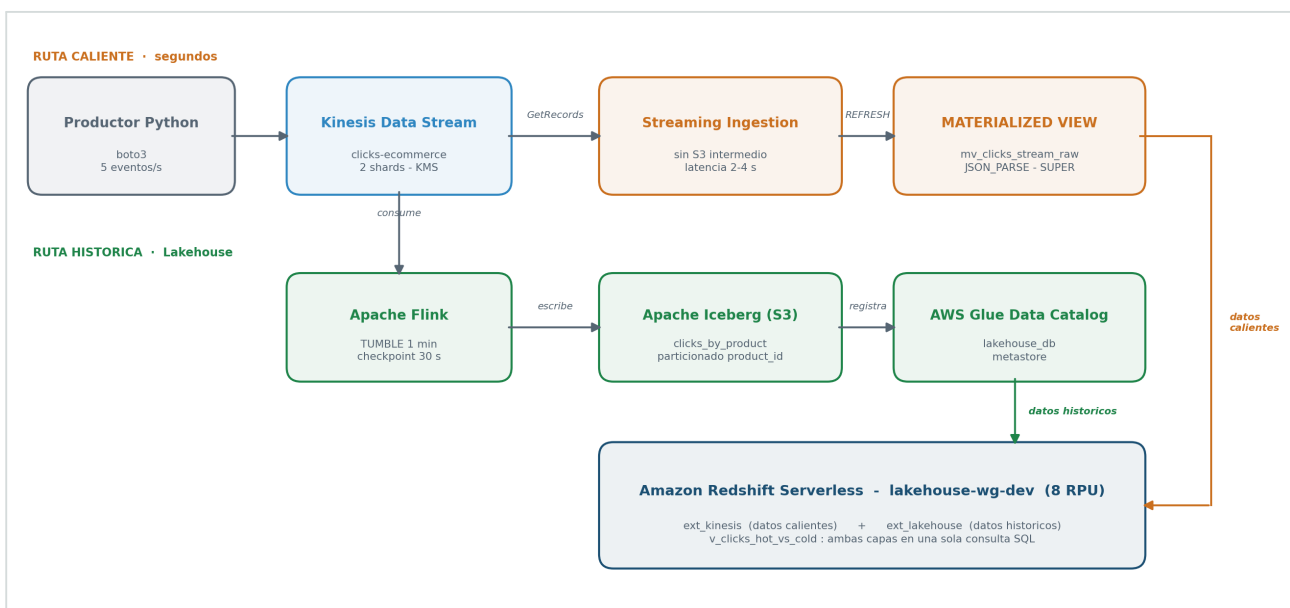


Figura 1. Arquitectura del pipeline. La ruta caliente (naranja) lleva los eventos de Kinesis a Redshift por Streaming Ingestion en segundos; la ruta historica (verde) los consolida en Iceberg via Flink. Redshift consulta ambas capas en la misma sesion SQL.

Componentes construidos

- **Streaming Ingestion:** esquema externo `ext_kinesis` que puentea Kinesis con Redshift.
- **Vista materializada** `mv_clicks_stream_raw` con refresco incremental y proteccion frente a *schema drift*.
- **Modelado del JSON** a tipos nativos: cinco campos extraidos y casteados sin errores.
- **Integracion Lakehouse:** esquema externo `ext_lakehouse` sobre el Glue Data Catalog, leyendo la tabla Iceberg.
- **Consulta federada** que combina datos calientes e historicos.
- **Seguridad:** rol de analitica con privilegios acotados y usuario con autentificacion IAM.
- **Monitoreo:** consultas de sistema para medir el lag de ingesta por shard.

2. Infraestructura como codigo

Todo el despliegue esta declarado en Terraform, en el modulo `infra/environments/dev/modules/redshift/`. Se eligio **Redshift Serverless** en lugar de un cluster provisionado por una razon economica concreta: Serverless factura por RPU-segundo de computo efectivamente consumido, mientras que un cluster provisionado factura mientras existe, aunque nadie lo consulte.

Recurso Terraform	Funcion
<code>aws_redshiftserverless_namespace</code>	Capa logica: base de datos, credenciales y permisos. La contraseña del usuario admin la genera y rota AWS Secrets Manager (<code>manage_admin_password</code>), de modo que no existe ningun secreto en el codigo ni en el state.
<code>aws_redshiftserverless_workgroup</code>	Capa de computo: 8 RPU (el minimo), tres subredes privadas y sin acceso publico.
<code>aws_iam_role.redshift_role</code>	Rol de servicio que Redshift asume desde SQL en las sentencias <code>CREATE EXTERNAL SCHEMA</code> .
<code>aws_iam_role_policy.redshift_kinesis_read</code>	<code>kinesis:DescribeStream</code> , <code>GetShardIterator</code> , <code>GetRecords</code> , <code>ListShards</code> y <code>kms:Decrypt</code> acotado por <code>ViaService</code> .
<code>aws_iam_role_policy.redshift_glue_lakehouse</code>	Lectura del Glue Data Catalog y de los archivos Parquet del Lakehouse en S3.
<code>aws_security_group.redshift_sg</code>	Puerto 5439 restringido al CIDR de la VPC.

Un requisito que conviene conocer de antemano

Redshift Serverless exige subredes en **al menos tres Zonas de Disponibilidad distintas**. La VPC del proyecto tenia dos, de modo que el modulo de red se amplio a tres antes de poder desplegar. Es un detalle que no aparece en la documentacion introductoria y que hace fallar el `app ly` sin un mensaje demasiado explicito.

Permiso de KMS: por que aparece en la politica

El Kinesis Data Stream esta cifrado en reposo con KMS (alias/`aws/kinesis`). Sin la accion `kms:Decrypt`, Redshift obtiene los registros pero no puede descifrarlos, y la ingesta falla de una forma que parece un problema de permisos de Kinesis cuando en realidad es de cifrado. La politica la incluye con una condicion `ViaService` que limita su uso exclusivamente a operaciones de Kinesis en la region.

```
{
  "Sid": "DecryptKinesisRecords",
  "Effect": "Allow",
  "Action": ["kms:Decrypt"],
  "Resource": "*",
  "Condition": {
    "StringEquals": { "kms:ViaService": "kinesis.us-east-1.amazonaws.com" }
  }
}
```

3. Ingesta directa desde Kinesis

3.1 Esquema externo

La sentencia crea el puente entre Kinesis y Redshift. No hay bucket intermedio ni proceso de carga: Redshift lee los shards directamente mediante la API del stream.

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS ext_kinesis
FROM KINESIS
IAM_ROLE 'arn:aws:iam::010798385513:role/redshift-serverless-dev';
```

3.2 Vista materializada de aterrizaje

Esta vista es el punto de entrada de los datos y esta diseñada alrededor de una decision explicita: **no castea nada**. Guarda el payload como texto y como tipo SUPER, y marca cada registro con una bandera que indica si es JSON parseable.

Proteccion frente al schema drift

El enunciado senala el *schema drift* como error comun: si el productor cambia la estructura del JSON y la vista es rigida, la ingesta falla o produce nulos silenciosos. La estrategia adoptada evita ambos escenarios. CAN_JSON_PARSE evalua cada registro antes de intentar extraer nada; un registro corrupto **entra igual** a la vista, queda marcado como invalido y es visible en una vista de cuarentena. El casteo a tipos nativos ocurre despues, y solo sobre registros ya validados. La ingesta nunca se rompe, y el dato problematico no se pierde: queda disponible para inspeccion.

```
CREATE MATERIALIZED VIEW mv_clicks_stream_raw
DISTSTYLE EVEN
SORTKEY (approximate_arrival_timestamp)
AUTO REFRESH NO
AS
SELECT
  -- Metadatos que aporta Kinesis en cada registro
  kinesis.approximate_arrival_timestamp,
  kinesis.shard_id,
  kinesis.sequence_number,
  kinesis.partition_key,
  kinesis.refresh_time,

  -- Bandera de validez: permite detectar drift sin perder el dato
  CAN_JSON_PARSE(FROM_VARBYTE(kinesis.kinesis_data, 'utf-8')) AS es_json_valido,

  -- Payload crudo, util para auditar registros que no parsean
  FROM_VARBYTE(kinesis.kinesis_data, 'utf-8') AS payload_texto,

  -- Payload semiestructurado (SUPER) para navegacion con notacion punto
  CASE
    WHEN CAN_JSON_PARSE(FROM_VARBYTE(kinesis.kinesis_data, 'utf-8'))
    THEN JSON_PARSE(FROM_VARBYTE(kinesis.kinesis_data, 'utf-8'))
    ELSE NULL
  END AS payload
FROM ext_kinesis."clicks-ecommerce" AS kinesis;

REFRESH MATERIALIZED VIEW mv_clicks_stream_raw;
```

```

SELECT COUNT(*) AS total_eventos,
       SUM(CASE WHEN es_json_valido THEN 1 ELSE 0 END) AS json_validos,
       MIN(approximate_arrival_timestamp) AS primer_evento,
       MAX(approximate_arrival_timestamp) AS ultimo_evento
FROM mv_clicks_stream_raw;

```

total_eventos	json_validos	primer_evento	ultimo_evento
1736	1736	2026-09-09 22:39:57.517	2026-09-09 22:49:56.638

Query ID: 206232 Elapsed time: 310 ms Total rows: 1

Figura 2. Resultado de la ingesta: 1.736 eventos recibidos desde Kinesis, los 1.736 con JSON valido. La ventana temporal cubre los diez minutos de emision del productor.

4. Modelado del JSON a tipos nativos

La vista tipada extrae cinco campos del evento y los convierte a tipos nativos de Redshift. Además calcula una métrica que resulta útil para el monitoreo: la diferencia entre el momento en que el evento ocurrió (*event time*, dentro del JSON) y el momento en que Kinesis lo recibió (*processing time*).

```

CREATE OR REPLACE VIEW public.v_clicks_tipado AS
SELECT
  -- JSON_EXTRACT_PATH_TEXT es determinista y no deja comillas residuales,
  -- a diferencia del cast directo de SUPER a VARCHAR.
  json_extract_path_text(k.payload_texto, 'event_id') AS event_id,
  json_extract_path_text(k.payload_texto, 'user_id') AS user_id,
  json_extract_path_text(k.payload_texto, 'event_type') AS event_type,
  json_extract_path_text(k.payload_texto, 'product_id') AS product_id,

  -- Se castea a TIMESTAMP (no TIMESTAMPTZ) para que DATEDIFF pueda compararlo
  -- con approximate_arrival_timestamp, que Kinesis entrega sin zona horaria.
  -- Ambos valores estan en UTC.
  json_extract_path_text(k.payload_texto, 'timestamp')::TIMESTAMP AS event_timestamp,

  k.approximate_arrival_timestamp,

  -- Latencia real de la ruta productor -> stream, en segundos
  DATEDIFF(second,
    json_extract_path_text(k.payload_texto, 'timestamp')::TIMESTAMP,
    k.approximate_arrival_timestamp) AS latencia_ingesta_seg,

  k.shard_id
FROM public.mv_clicks_stream_raw AS k
WHERE k.es_json_valido = TRUE; -- los invalidos quedan fuera, pero no se pierden

```

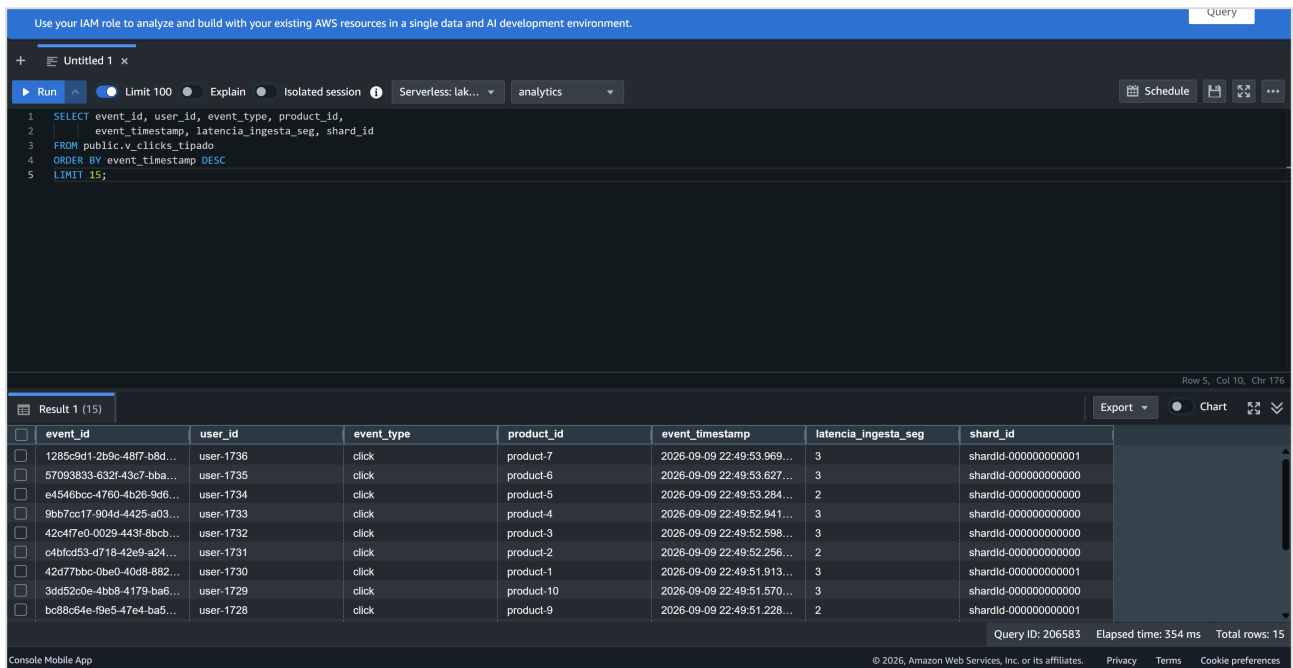


Figura 3. El JSON convertido en columnas con tipos reales. La columna `latencia_ingesta_seg` muestra consistentemente 2 y 3 segundos, y `shard_id` confirma que la lectura cubre los dos shards del stream.

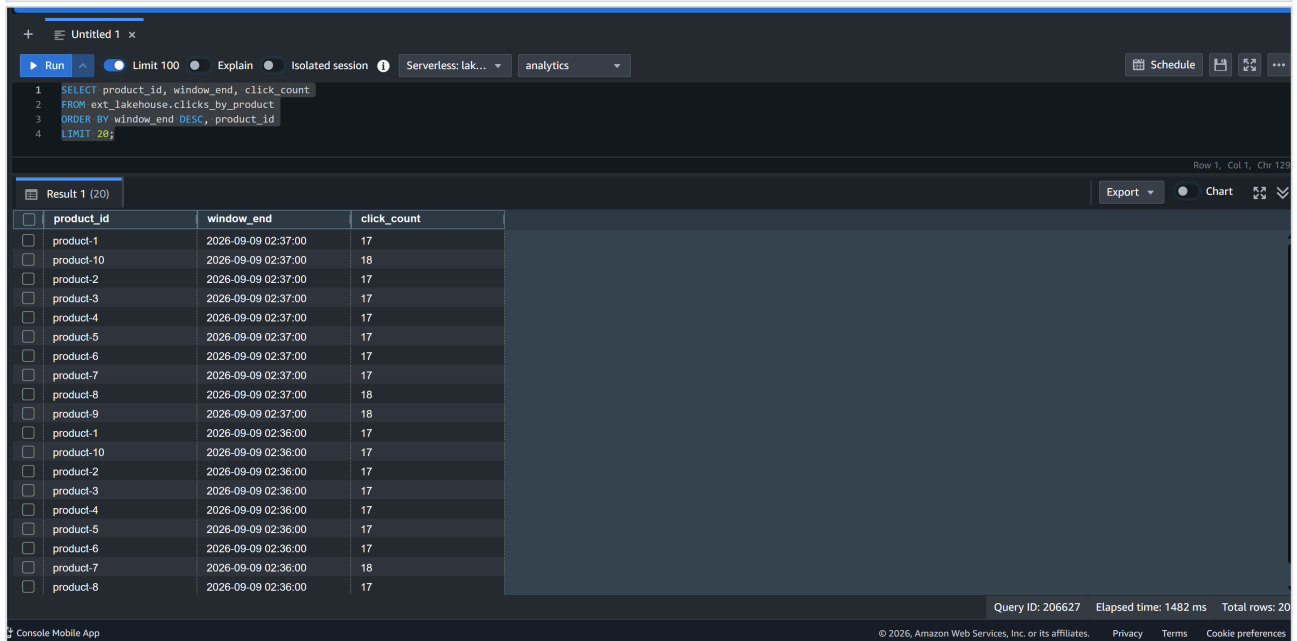
Dos incompatibilidades de tipos que vale documentar

- 1. El alias `raw` es palabra reservada.** Redshift rechaza `FROM mv_clicks_stream_raw AS raw` con un error de sintaxis que apunta al nombre de la función y no al alias, lo que despista. Se resolvió renombrando el alias a `k`.
- 2. `TIMESTAMP` frente a `TIMESTAMPZ`.** El campo `approximate_arrival_timestamp` que entrega Kinesis es `TIMESTAMP` sin zona horaria. Castear el timestamp del evento a `TIMESTAMPZ` hace que `DATEDIFF` falle porque no existe una firma que mezcle ambos tipos. Como los dos valores están en UTC, la solución correcta es unificar en `TIMESTAMP`.

5. Integracion con el Lakehouse

El segundo esquema externo apunta al Glue Data Catalog creado en el modulo anterior. Redshift Spectrum lee la tabla Apache Iceberg consultando sus metadatos en Glue y sus archivos Parquet en S3, **sin copiar ni mover un solo byte**. El dato sigue viviendo en el Lakehouse; Redshift solo lo consulta.

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS ext_lakehouse
FROM DATA CATALOG
DATABASE 'lakehouse_db'
IAM_ROLE 'arn:aws:iam::010798385513:role/redshift-serverless-dev'
REGION 'us-east-1';
```



Query ID: 206627 Elapsed time: 1482 ms Total rows: 20

product_id	window_end	click_count
product-1	2026-09-09 02:37:00	17
product-10	2026-09-09 02:37:00	18
product-2	2026-09-09 02:37:00	17
product-3	2026-09-09 02:37:00	17
product-4	2026-09-09 02:37:00	17
product-5	2026-09-09 02:37:00	17
product-6	2026-09-09 02:37:00	17
product-7	2026-09-09 02:37:00	17
product-8	2026-09-09 02:37:00	18
product-9	2026-09-09 02:37:00	18
product-1	2026-09-09 02:36:00	17
product-10	2026-09-09 02:36:00	17
product-2	2026-09-09 02:36:00	17
product-3	2026-09-09 02:36:00	17
product-4	2026-09-09 02:36:00	17
product-5	2026-09-09 02:36:00	17
product-6	2026-09-09 02:36:00	17
product-7	2026-09-09 02:36:00	18
product-8	2026-09-09 02:36:00	17

Figura 4. La tabla Iceberg `clicks_by_product` consultada desde Redshift. Los valores de `click_count` son los que Flink calculo con ventanas `TUMBLE` de un minuto en el modulo anterior.

5.1 Consulta federada: datos calientes y datos historicos

Esta es la pieza central del checkpoint. La vista combina las dos capas: por un lado los clics que llegaron por el stream hace segundos, por otro el consolidado historico que Flink dejo en Iceberg. Ambos se resuelven en la misma sentencia SQL y en la misma sesion.

```

CREATE OR REPLACE VIEW public.v_clicks_hot_vs_cold AS
WITH caliente AS (
  -- Datos en vivo: llegan del stream con segundos de latencia
  SELECT
    product_id,
    COUNT(*) AS clicks_stream,
    MAX(event_timestamp) AS ultimo_click
  FROM public.v_clicks_tipado
  GROUP BY product_id
),
historico AS (
  -- Datos consolidados por Flink en la tabla Iceberg del Lakehouse
  SELECT
    product_id,
    SUM(click_count) AS clicks_historicos,
    COUNT(*) AS ventanas_registradas,
    MAX(window_end) AS ultima_ventana
  FROM ext_lakehouse.clicks_by_product
  GROUP BY product_id
)
SELECT
  COALESCE(c.product_id, h.product_id) AS product_id,
  COALESCE(c.clicks_stream, 0) AS clicks_stream,
  c.ultimo_click,
  COALESCE(h.clicks_historicos, 0) AS clicks_historicos,
  h.ventanas_registradas,
  h.ultima_ventana
FROM caliente c
FULL OUTER JOIN historico h ON c.product_id = h.product_id
-- Obligatorio: una vista que referencia tablas externas debe ser late-binding.
WITH NO SCHEMA BINDING;

```

Use your IAM role to analyze and build with your existing AWS resources in a single data and AI development environment.

Query

Untitled 1 x

Run Limit 100 Explain Isolated session Serverless: lak... analytics Schedule

```

1 SELECT * FROM public.v_clicks_hot_vs_cold ORDER BY clicks_stream DESC;
2

```

Row 2, Col 1, Chr 72

Result 1 (10)

product_id	clicks_stream	ultimo_click	clicks_historicos	ventanas_registradas	ultima_ventana
product-6	174	2026-09-09 22:49:53.627...	85	5	2026-09-09 02:37:00
product-4	174	2026-09-09 22:49:52.941...	85	5	2026-09-09 02:37:00
product-2	174	2026-09-09 22:49:52.256...	85	5	2026-09-09 02:37:00
product-5	174	2026-09-09 22:49:53.284...	85	5	2026-09-09 02:37:00
product-3	174	2026-09-09 22:49:52.598...	85	5	2026-09-09 02:37:00
product-7	174	2026-09-09 22:49:53.969...	85	5	2026-09-09 02:37:00
product-1	173	2026-09-09 22:49:51.913...	84	5	2026-09-09 02:37:00
product-9	173	2026-09-09 22:49:51.228...	85	5	2026-09-09 02:37:00
product-8	173	2026-09-09 22:49:50.885...	85	5	2026-09-09 02:37:00
product-10	173	2026-09-09 22:49:51.570...	85	5	2026-09-09 02:37:00

Query ID: 206731 Elapsed time: 992 ms Total rows: 10

Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Figura 5. Consulta federada. Cada fila combina, para un mismo producto, los clics en vivo del stream (*clicks_stream*, *ultimo_click*) con el consolidado del Lakehouse (*clicks_historicos*, *ventanas_registradas*, *ultima_ventana*).

Por que hace falta WITH NO SCHEMA BINDING

Redshift valida las dependencias de una vista en el momento de crearla. Pero un esquema externo apunta a metadatos que viven **fuera** de Redshift — en el Glue Data Catalog — y que pueden evolucionar de forma independiente. El *late binding* difiere esa validacion hasta el momento de la consulta. Es coherente con el diseno de Iceberg: la tabla puede cambiar su esquema o su particionado sin romper lo que esta aguas abajo.

6. Seguridad

El acceso analítico se canaliza por un rol específico con privilegios estrictamente de lectura, siguiendo el principio de mínimo privilegio que ya se aplicó a los roles de Flink y Firehose en los módulos anteriores.

```
-- Rol de analítica con privilegios estrictamente de lectura
CREATE ROLE analytics_reader;

-- Acceso a las capas de consumo, no a la fuente cruda del stream
GRANT USAGE ON SCHEMA public TO ROLE analytics_reader;
GRANT USAGE ON SCHEMA ext_lakehouse TO ROLE analytics_reader;

GRANT SELECT ON public.v_clicks_tipado TO ROLE analytics_reader;
GRANT SELECT ON public.v_clicks_hot_vs_cold TO ROLE analytics_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA ext_lakehouse TO ROLE analytics_reader;

-- Usuario de analítica sin contraseña: la identidad la valida IAM
CREATE USER analyst_user PASSWORD DISABLE;
GRANT ROLE analytics_reader TO analyst_user;
```

Dos decisiones de diseño

- **El esquema ext_kinesis queda fuera del rol.** No se otorga USAGE sobre el esquema de ingesta. Un analista consulta datos modelados y validados, no la fuente cruda del stream: eso evita que una consulta mal formada sobre VARBYTE devuelva resultados engañosos, y reduce la superficie expuesta.
- **PASSWORD DISABLE fuerza autenticación federada.** El usuario no tiene contraseña propia: la identidad la valida IAM. No hay credencial que robar, filtrar ni rotar, y el acceso queda auditado en CloudTrail junto con el resto de la actividad de la cuenta.

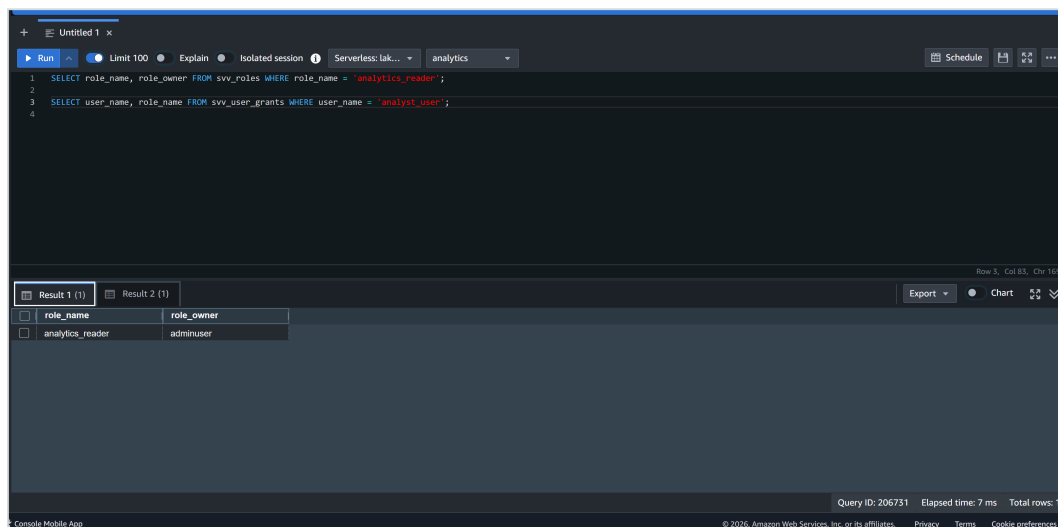


Figura 6. Rol `analytics_reader` creado, con `adminuser` como propietario.

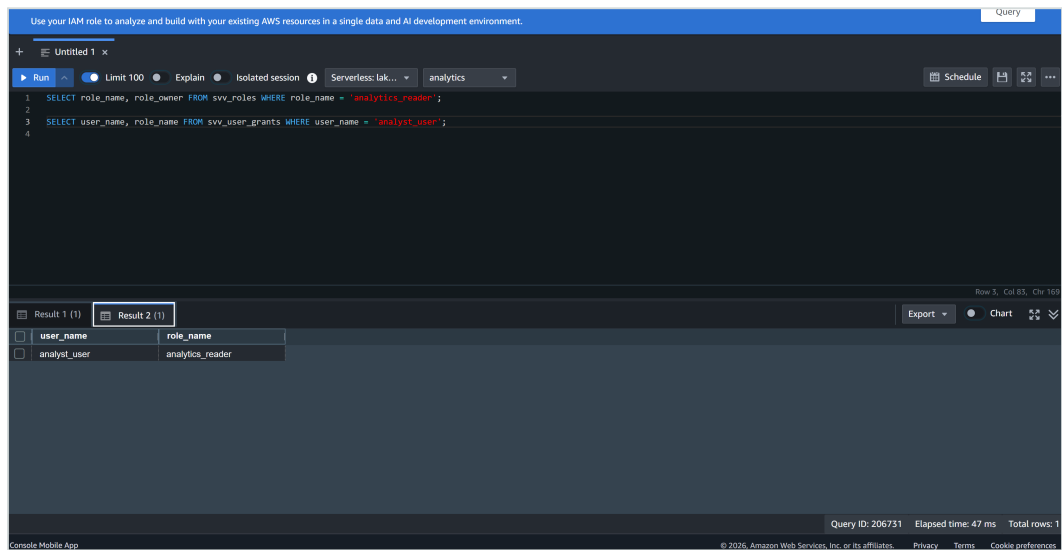


Figura 7. El usuario `analyst_user` tiene asignado el rol `analytics_reader`.

7. Estrategia de refresco

El refresco de una vista materializada sobre Streaming Ingestion es **incremental**: Redshift lee unicamente los registros posteriores al ultimo `sequence_number` procesado en cada shard, y no reprocesa el historico. Ese comportamiento es visible en la Figura 9, donde cada shard mantiene su propia posicion de lectura.

El compromiso entre latencia y costo

El enunciado advierte sobre intentar refrescar cada segundo. El razonamiento es economico ademas de tecnico: cada refresco consume RPU, y en Redshift Serverless la facturacion es por RPU-segundo con un minimo por operacion. Un refresco por segundo mantiene el cluster ocupado de forma permanente y multiplica el gasto sin aportar valor si el negocio no consume la informacion a ese ritmo.

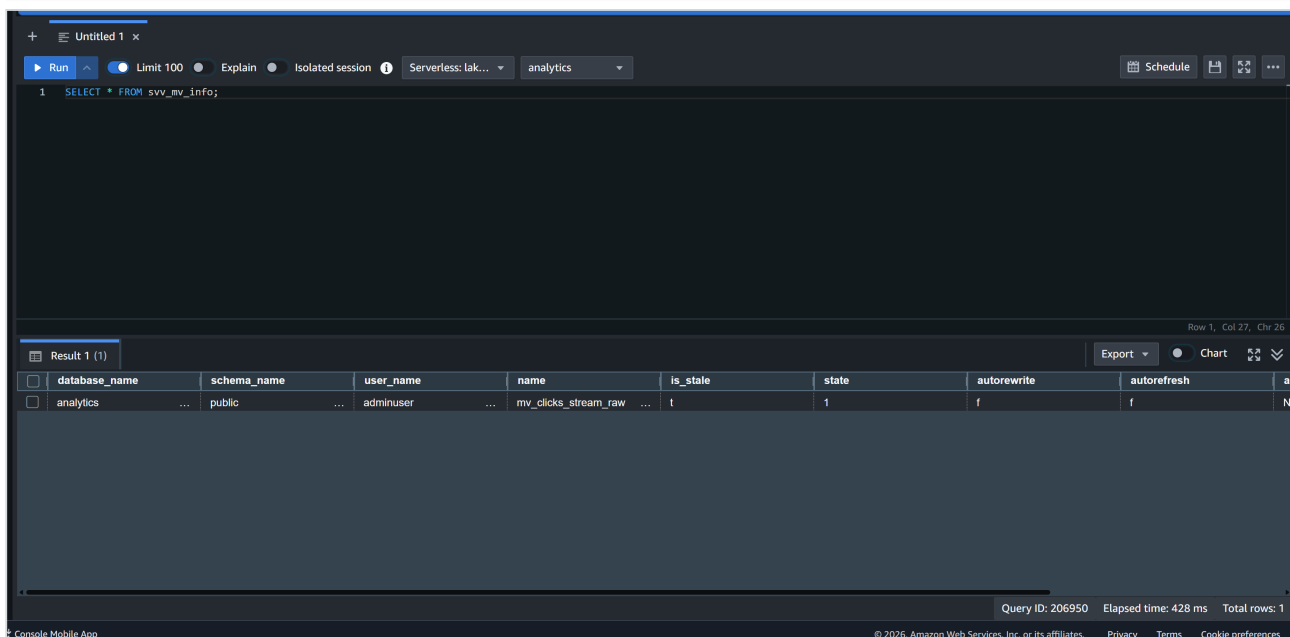
Intervalo	Latencia percibida	Consumo de RPU	Cuando conviene
1 segundo	Casi nula	Muy alto y constante	Deteccion de fraude, trading, alarmas criticas
30–60 segundos	Aceptable para decisiones operativas	Acotado y predecible	Este caso de uso: ranking de productos por actividad reciente
5 minutos o mas	Alta	Minimo	Reportes tacticos que ya cubre la ruta del Lakehouse

Para este pipeline se adopto un intervalo del orden de **60 segundos**. Ninguna decision comercial sobre un ranking de productos se toma con granularidad menor, de modo que refrescar mas seguido solo agregaria costo. Durante la demostracion la vista se mantuvo en `AUTO REFRESH NO` para controlar de forma explicita el momento de cada carga y poder medir la latencia con precision.

```
-- Refresco manual: control total del momento de carga (usado en la demostracion)
REFRESH MATERIALIZED VIEW mv_clicks_stream_raw;

-- Refresco automatico gobernado por Redshift, para operacion continua
ALTER MATERIALIZED VIEW mv_clicks_stream_raw AUTO REFRESH YES;

-- Volver a manual al terminar, para no consumir RPU en segundo plano
ALTER MATERIALIZED VIEW mv_clicks_stream_raw AUTO REFRESH NO;
```



database_name	schema_name	user_name	name	is_stale	state	autorewrite	autorefresh
analytics	public	adminuser	mv_clicks_stream_raw	t	1	f	f

Figura 8. Estado de la vista materializada: esquema public, propietario adminuser y autorefresh en 'f', coherente con la estrategia de refresco controlado adoptada durante la prueba.

8. Monitoreo del lag de ingesta

El enunciado pide consultas de sistema para medir el lag. Se abordó desde dos ángulos complementarios: la vista de sistema que expone el estado interno del scan, y una medición calculada sobre los propios datos.

8.1 Estado del scan por shard

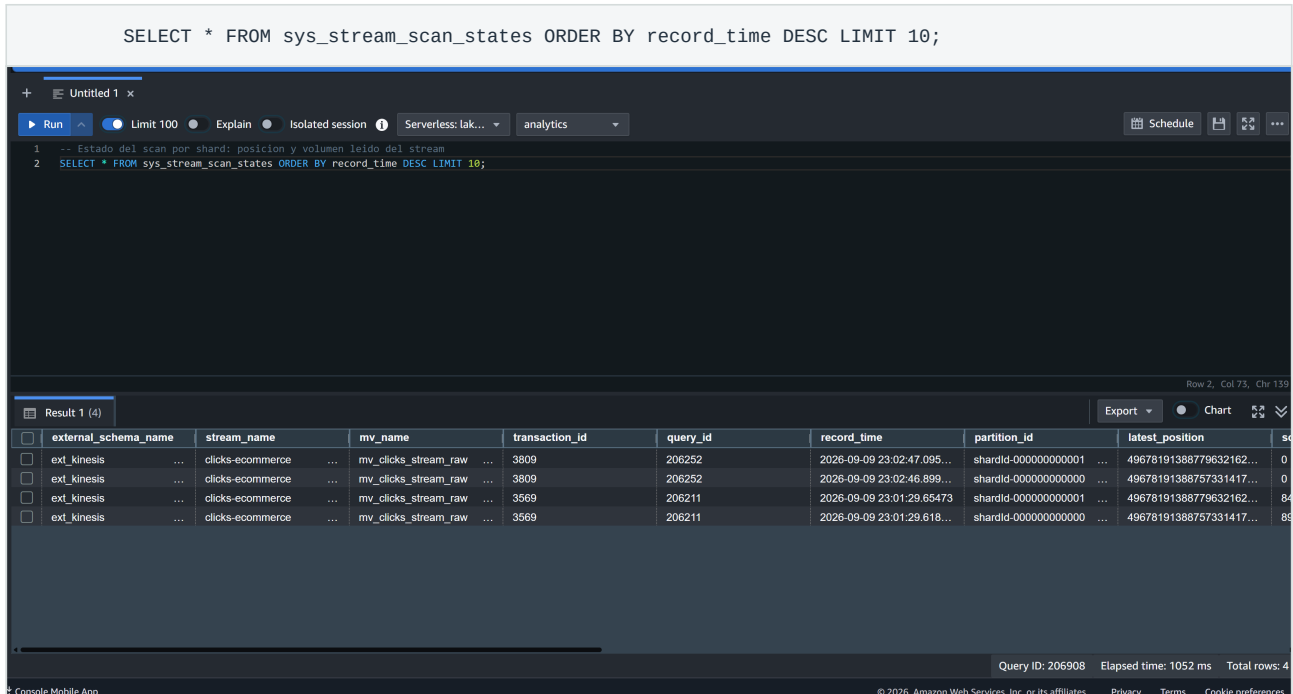


Figura 9. Estado del scan. Se observa el esquema `ext_kinesis` leyendo el stream `clicks-ecommerce` hacia la vista `mv_clicks_stream_raw`, con la posición de lectura (`latest_position`) registrada de forma independiente para cada uno de los dos shards.

8.2 Lag de punta a punta medido sobre los datos

Esta consulta responde a la pregunta que realmente importa al negocio: cuánto tarda un evento desde que ocurre hasta que es consultable en SQL.

```
SELECT
    COUNT(*) AS eventos_ingridos,
    MIN(latencia_ingesta_seg) AS lag_min_seg,
    ROUND(AVG(latencia_ingesta_seg), 2) AS lag_promedio_seg,
    MAX(latencia_ingesta_seg) AS lag_max_seg,
    MAX(approximate_arrival_timestamp) AS ultimo_evento_recibido
FROM public.v_clicks_tipado;
```

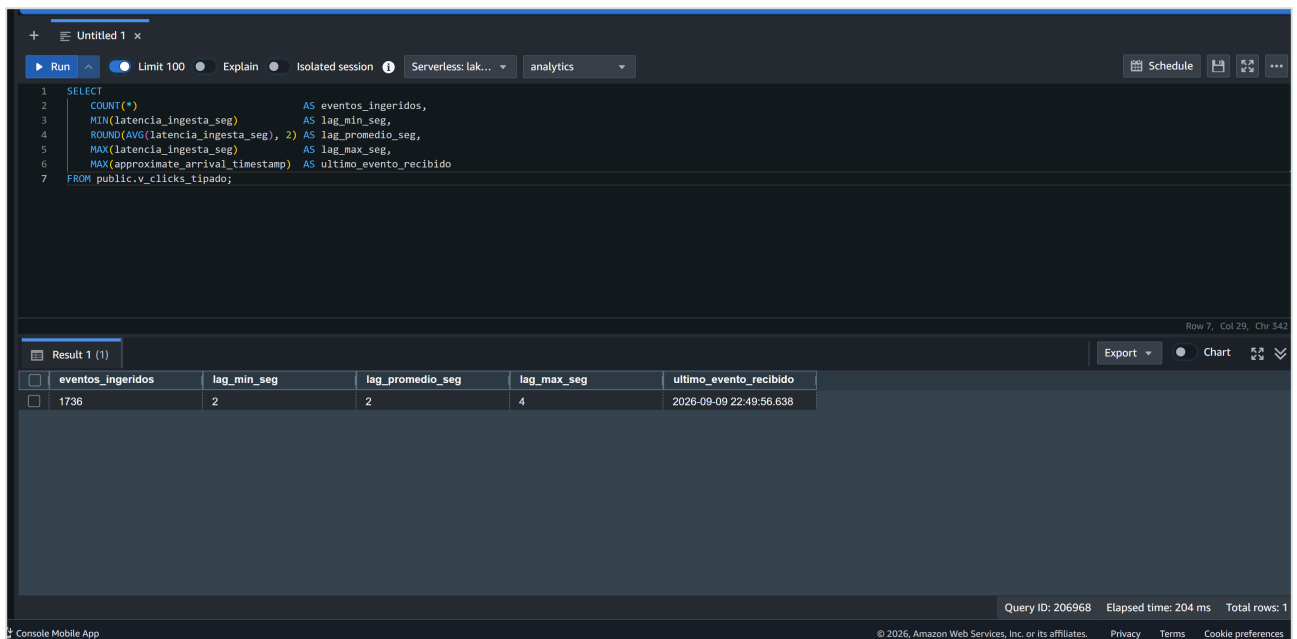


Figura 10. Lag de ingesta sobre los 1.736 eventos: minimo 2 segundos, promedio 2 segundos, maximo 4 segundos.

Lectura del resultado

Una latencia estable entre 2 y 4 segundos, sin cola larga, indica que la ruta no acumula retraso: Redshift consume los shards al ritmo al que el productor escribe. A modo de contraste, la misma informacion por la ruta del Lakehouse requiere que cierre la ventana de un minuto y que se complete el checkpoint de Flink que confirma el commit en Iceberg. Esa es exactamente la brecha que Streaming Ingestion viene a cubrir.

9. Metadatos de Iceberg en el catalogo

La tabla que Redshift consulta por Spectrum es la misma que Flink escribio en el modulo anterior. La consola de AWS Glue confirma el formato y el estado del catalogo.

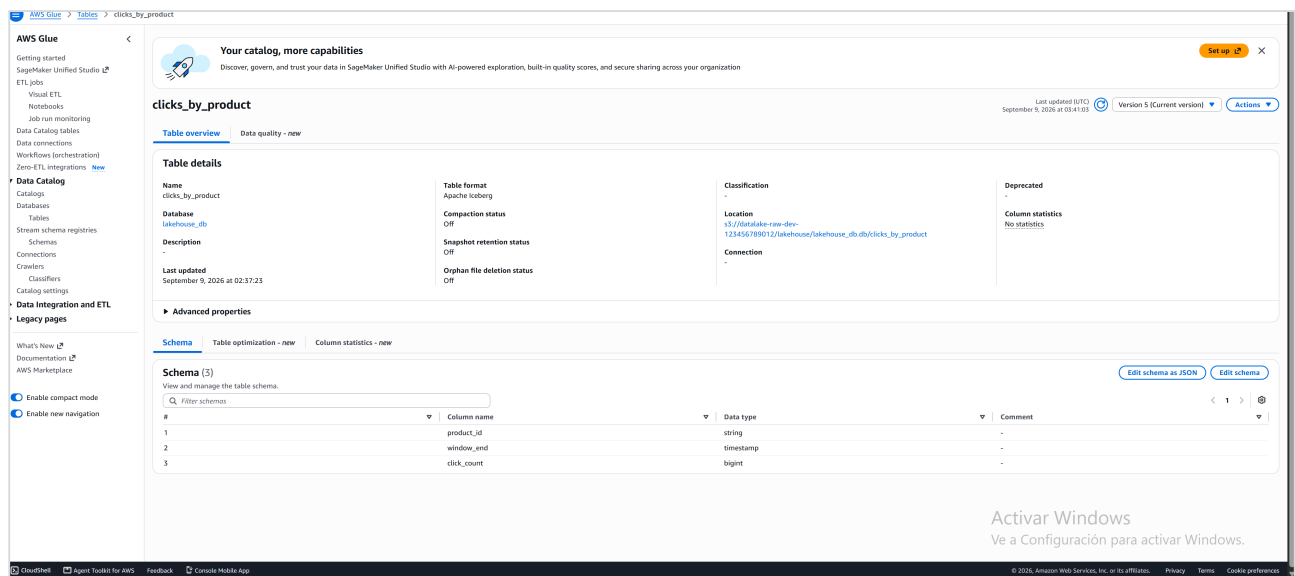


Figura 11. La tabla clicks_by_product en el Glue Data Catalog: formato Apache Iceberg, base de datos lakehouse_db, version 5 del metadata (cinco commits confirmados por Flink) y el esquema de tres columnas que consume Redshift.

El indicador Version 5 merece una lectura: cada version corresponde a un commit de metadatos que Flink confirmo al completar un checkpoint. Es la trazabilidad transaccional de Iceberg, y es la razon por la que Redshift puede leer la tabla con garantia de consistencia mientras Flink sigue escribiendo.

10. Verificacion contra los criterios de evaluacion

Criterio	Evidencia	Estado
Funcionalidad Los datos llegan de Kinesis a Redshift sin errores de casting	1.736 eventos ingeridos, 1.736 con JSON valido y cero registros en cuarentena. Cinco campos casteados a tipos nativos (Figuras 2 y 3).	Cumplido
Integracion Consultar Iceberg y el stream en la misma sesion SQL	La vista v_clicks_hot_vs_cold devuelve, por producto, los clics del stream junto a las 5 ventanas historicas de la tabla Iceberg (Figuras 4 y 5).	Cumplido
Seguridad Autenticacion IAM entre servicios	Rol de servicio con permisos acotados por recurso; rol analytics_reader de solo lectura; usuario con PASSWORD DISABLE (Figuras 6 y 7).	Cumplido
Claridad SQL comentado y buenas practicas de nombrado	Script unico organizado en 7 bloques, con prefijos consistentes: ext_ para esquemas externos, mv_ para vistas materializadas y v_ para vistas logicas.	Cumplido

11. Conclusiones

La incorporacion de Redshift completa el pipeline con una capa que faltaba: la consulta de baja latencia sobre datos que todavia estan llegando. Los numeros lo respaldan — 2 segundos de latencia media contra los minutos que impone el cierre de ventana mas el checkpoint de la ruta Flink/Iceberg.

Pero el resultado mas interesante no es la velocidad sino la **convivencia**. Las dos rutas no compiten: resuelven preguntas distintas. El stream responde *que esta pasando ahora*; el Lakehouse responde *que viene pasando*, con la consistencia transaccional y la eficiencia de lectura que dan Iceberg y su particionado. Que ambas se consulten con una sola sentencia SQL, sin mover datos entre sistemas, es lo que convierte a esto en una arquitectura Lakehouse y no en dos silos con un informe pegado encima.

En el plano operativo, el trabajo dejo tres aprendizajes concretos: el requisito de tres Zonas de Disponibilidad de Redshift Serverless, la necesidad de kms:Decrypt para leer un stream cifrado, y la obligatoriedad del *late binding* en vistas que tocan tablas externas. Ninguno es evidente antes de encontrarlo, y los tres estan documentados en este informe y comentados en el script.

Artefacto	Ubicacion en el repositorio
Script SQL completo (7 bloques comentados)	app/redshift/checkpoint6_redshift.sql
Modulo Terraform de Redshift	infra/environments/dev/modules/redshift/
Productor de eventos	app/producers/producer_kinesis_stream.py
Job de Flink hacia Iceberg (modulo 5)	app/flink/iceberg_processor.py